Cloud Native AI Agents with ADK, A2A, and kagent



Fon - Kamolphan Liwprasert
Data Team Lead @ AIMET.tech

Cloud Native AI Agents with ADK, A2A, and kagent

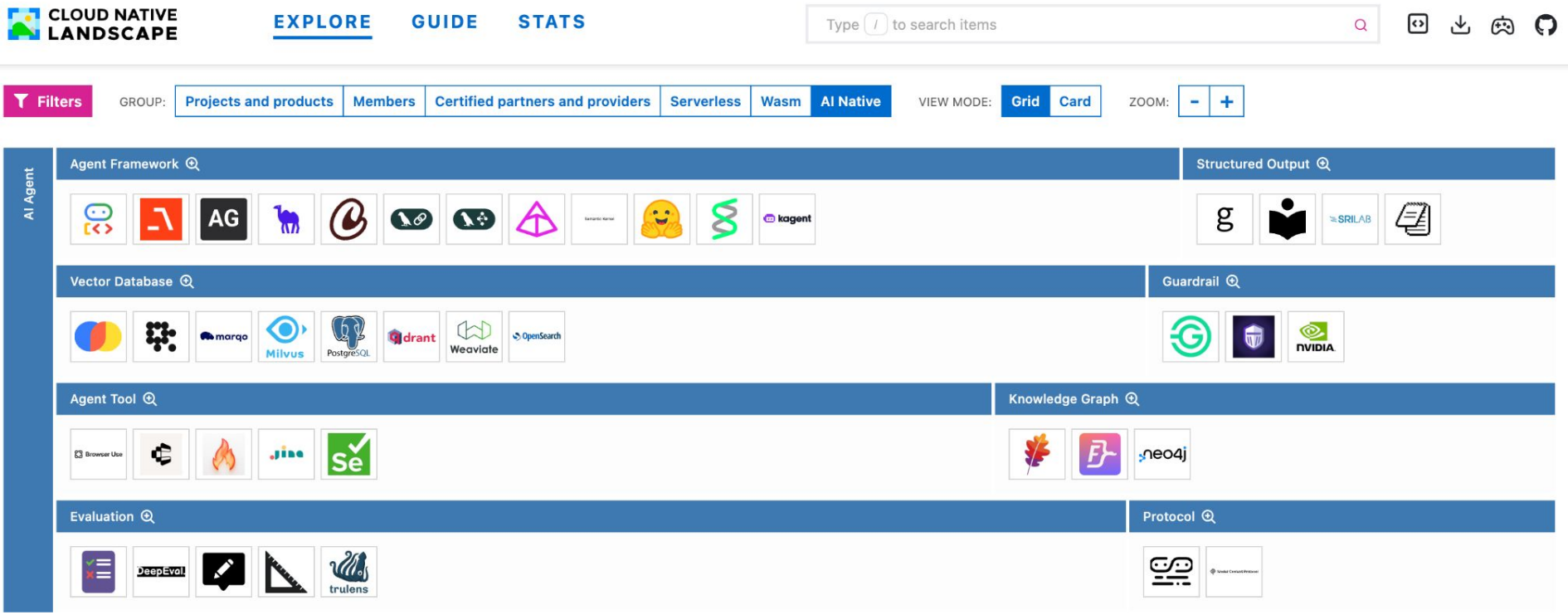


Fon - Kamolphan Liwprasert
Data Team Lead @ AIMET.tech

CNCF Landscape

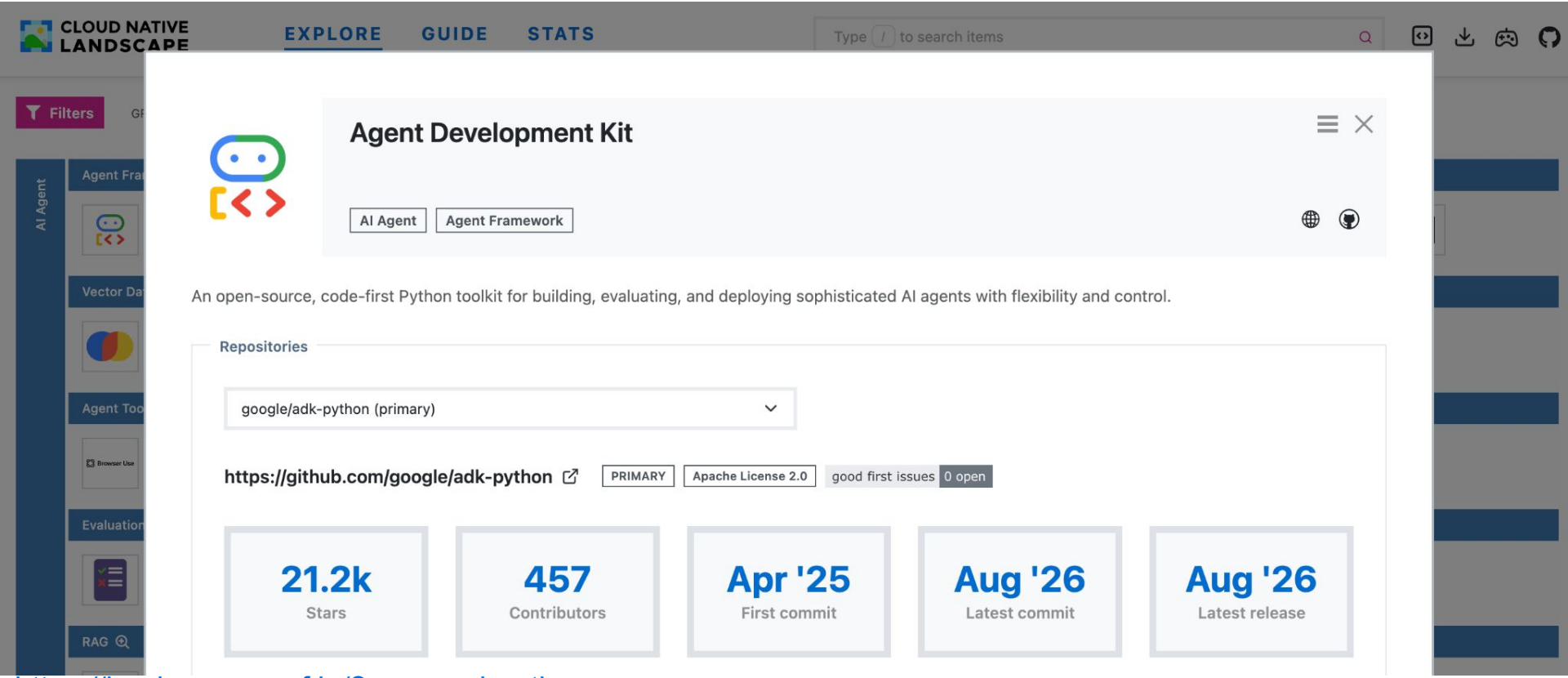


CNCF Landscape - AI Native



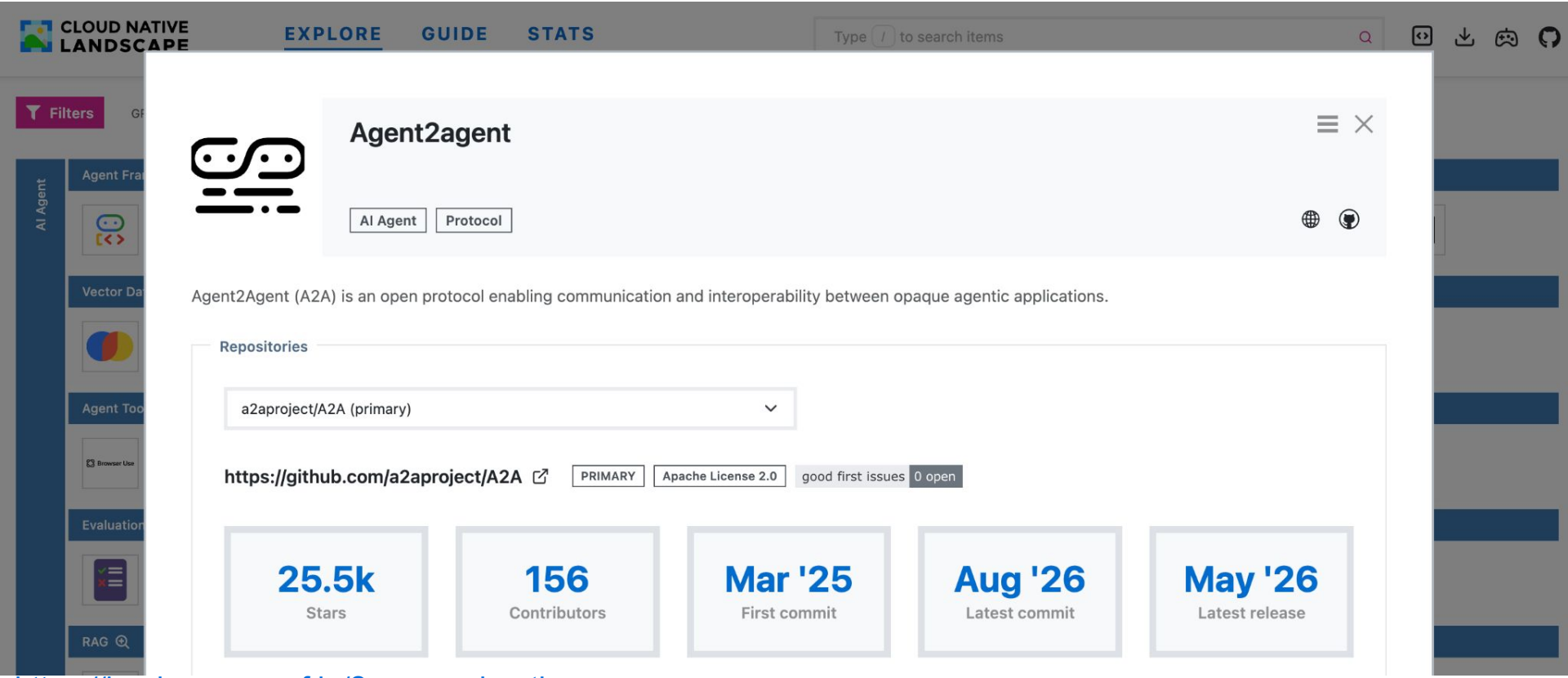
https://landscape.cncf.io/?group=ai-native

CNCF Landscape - AI Native



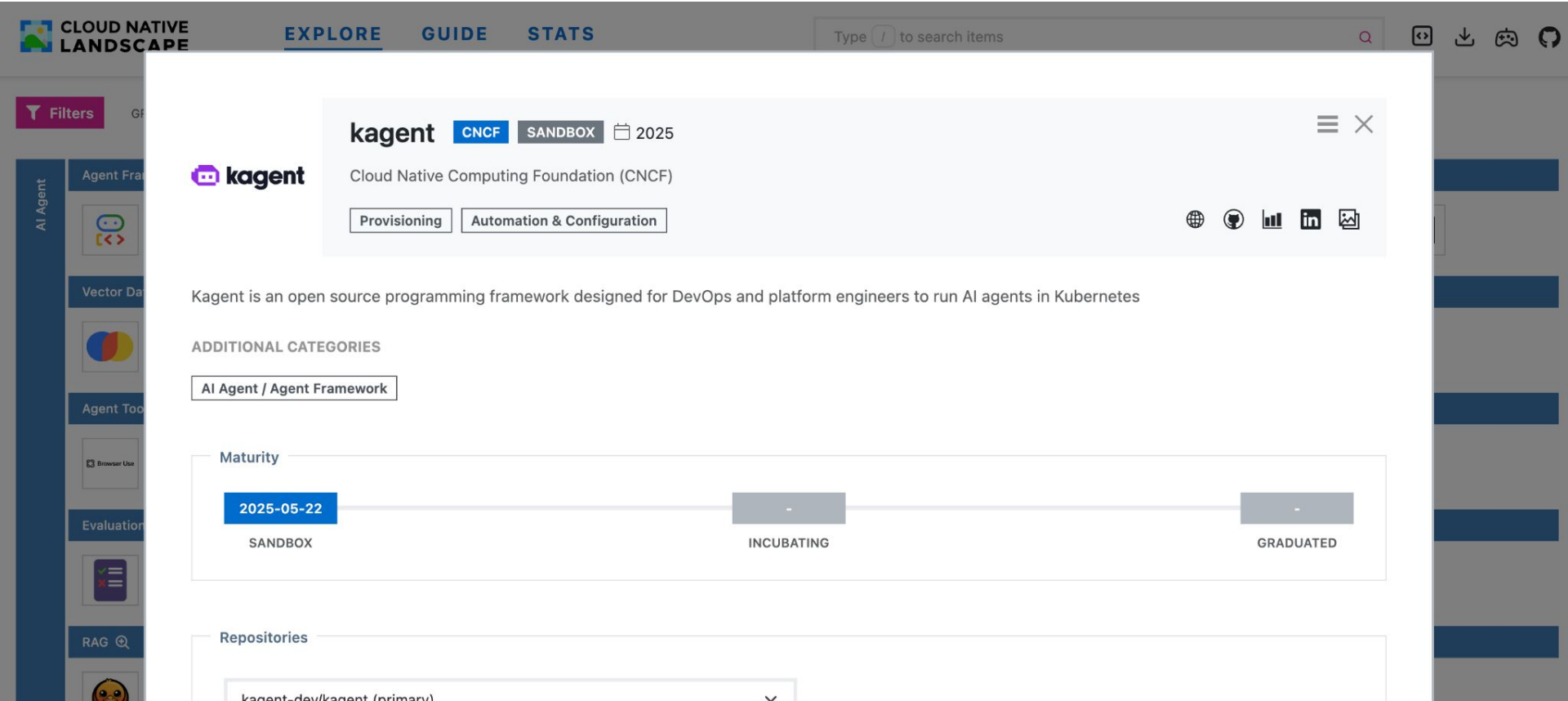
https://landscape.cncf.io/?group=ai-native

CNCF Landscape - AI Native



https://landscape.cncf.io/?group=ai-native

CNCF Landscape - AI Native



Challenges of Cloud-Native agents at Scale

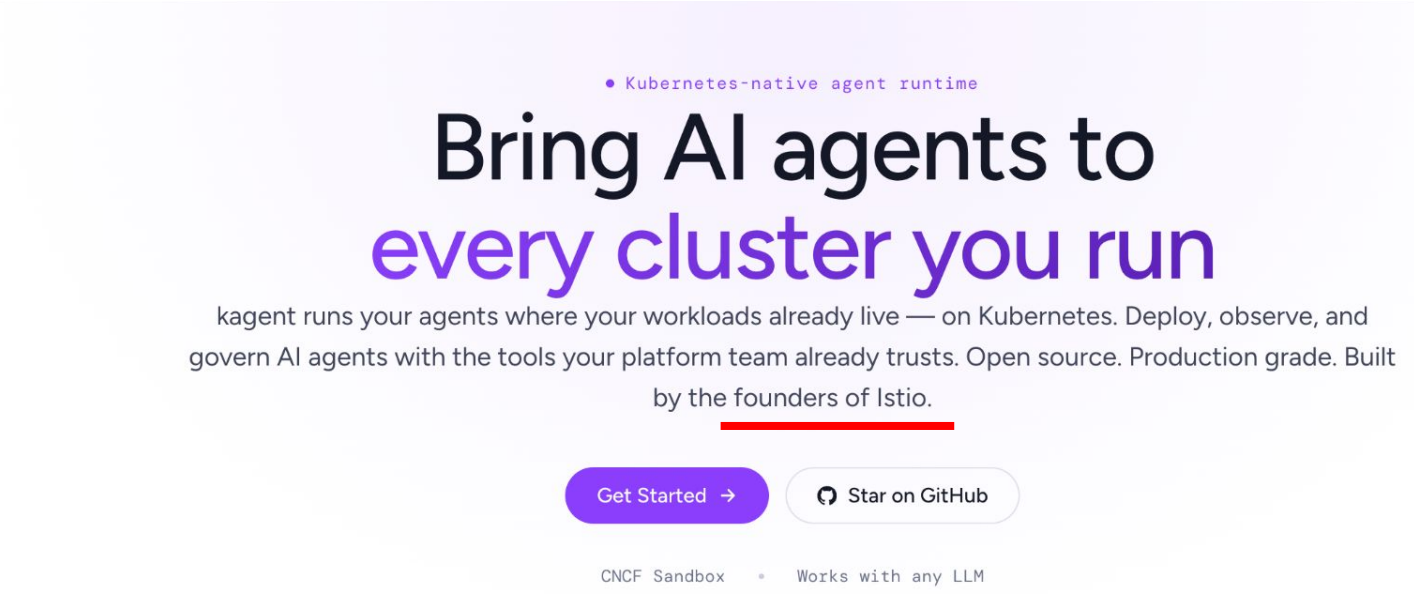
Challenges 🔥

- **Sandboxing:** Secure, isolated environments for code execution.
- **Lifecycle Management:** Continuous deployment, scaling, state persistence, and graceful termination.
- **Multi-tenancy:** Safely running multiple agents on shared cluster resources.
- **Role-Based Access Control:** Strict permissions for resources and APIs.
- **Tool Connectivity:** Secure and reliable connections to external databases and APIs.

kagent



kagent : Kubernetes-native agent runtime



https://kagent.dev

Why kagent?

Declarative agents

Define agents as Kubernetes CRDs — versioned in Git, reviewed in PRs, rolled out with the same tools you already use.

Bring your own everything

Any LLM, any framework, any tool. Native MCP, A2A, and OpenAI-compatible endpoints — no lock-in, no rewrites.

Observable by default

OpenTelemetry traces, Prometheus metrics, structured logs. See every prompt, every tool call, every token.

Standards-based

Native MCP, A2A, OpenTelemetry, and Kubernetes APIs. No proprietary glue, no rewrite tax later.

Zero-trust ready

Run on top of Istio or Ambient Mesh. mTLS, fine-grained RBAC, and policy-driven egress for agent traffic.

NVIDIA NemoClaw

Built-in security and privacy guardrails via NVIDIA NemoClaw. Run Nemotron locally or route to cloud models — with policy enforcement on every call.

Agent Substrate

Run declarative agents on Agent Substrate for fast startup, efficient resource usage, and secure sandboxed execution — a Kubernetes-native runtime for agents at scale.

https://kagent.dev

kagent

Public

Watch 23 | Fork 728 | Star 3.6k

main | 46 Branches | 168 Tags | Go to file T | Add file | Code

Charlesthebird and claude refactor(ui): rename the extension framework... 9e246fd · 12 hours ago | 1,573 Commits

.agents/skills	feat: agent sandbox support (#1640)	4 months ago
.claude/skills	feat: validate protobuf requests with protovalidate (#2563)	3 days ago
.devcontainer	test: add AgentInstance interaction e2e (#2514)	last week
.githooks	cncf/incubation: add technical review first draft, update co...	9 months ago
.github	feat(ui): rewrite the web interface on Vite, and give conver...	2 days ago
contrib	refactor: simplify runtime images and add lifecycle e2e (#...	last week
design	chore: remove legacy ACP and controller runtime (#2565)	3 days ago
docs	feat: add API v2 MCP tasks gateway (#2576)	2 days ago
examples	test: add AgentInstance interaction e2e (#2514)	last week
go	refactor: remove legacy session and task services (#2598)	19 hours ago
helm	refactor(ui): rename the extension framework to App Exte...	12 hours ago
img	replace the outdated diagram with a gif from the website (...	2 months ago
proto	feat(ui): rewrite the web interface on Vite, and give conver...	2 days ago
python	refactor: remove legacy session and task services (#2598)	19 hours ago

About

Cloud Native Agentic AI | Discord: https://bit.ly/kagentdiscord

kagent.dev

agents | ai | cncf | devops | mcp

Readme

Apache-2.0 license

Code of conduct

Contributing

Security policy

Activity

Custom properties

3.6k stars

23 watching

728 forks

Report repository

Releases 165

v0.10.0-rc3 Latest

last week

+ 164 releases

https://github.com/kagent-dev/kagent

Compatibilities

Agent lifecycle via CRDs

Define, version, and roll out agents with kubectl and GitOps — the same workflow as every other workload.

Multi-runtime support

Go and Python ADK runtimes. Pick the language that fits, or mix both in the same cluster.

Agent Substrate runtime

Run agents on the substrate runtime — a WorkerPool + Actor model that delivers fast cold starts, low resource overhead, and secure isolation per agent.

BYO frameworks

LangGraph, CrewAI, Google ADK, or your own — bring any agent framework and kagent orchestrates it.

Long-term memory

Persistent vector-backed memory across sessions. Agents remember context, not just the last prompt.

Human-in-the-loop

Tool approval gates, agent-initiated questions, and cascading HITL — humans stay in control.

Agent-to-Agent (A2A)

Agents discover and invoke each other. Compose multi-agent workflows with first-class delegation.

Skills from Git

Load markdown knowledge from Git repos at startup. Agents learn your runbooks, not just generic docs.

Prompt templates

Reusable prompt fragments from ConfigMaps. DRY your system prompts across agents.

Context compaction

Auto-summarization of long histories. Agents stay coherent in extended conversations without blowing token budgets.

Sandbox & security

Agent sandboxing, RBAC, and security hardening out of the box. Run untrusted code safely.

Full observability

OTel tracing, Prometheus metrics, structured logs. See every prompt, every tool call, every token.

How to deploy and operate agents?

Agent deployment concept

Microservices Era

Core Unit	Service
Contract	REST / gRPC contract
Communication	Library call (In-process)
App Framework	<<various>>
Service Mesh	Sidecar / Mesh
Gateway	Ingress
Topology	Known at build time

AGENT ERA

Next-Gen Agent Era

Core Unit	Agent (CRD, Deployment, Service)
Contract	A2A agent card
Communication	A2A, MCP tool call (Over network)
App Framework	<<various>>
Service Mesh	agentgateway
Gateway	kgateway
Topology	Chosen dynamic at request time

Kubernetes native runtime

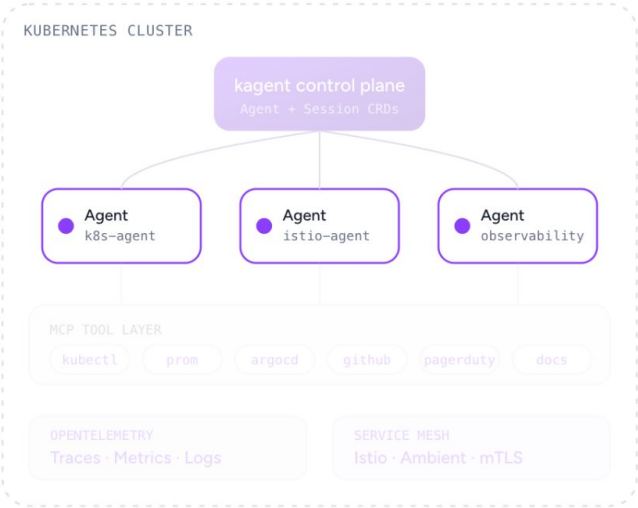
1. **Write an Agent CRD**
   Point at any LLM, attach the tools the agent needs (via MCP), and pick the cluster it should run in.

2. **kubectl apply -f agent.yaml**
   kagent provisions the runtime, wires up RBAC, and registers the agent with your service mesh.

3. **Tools and other agents**
   MCP servers expose your APIs as tools. A2A lets agents delegate to each other across the cluster.

4. **Traces, metrics, logs**
   Every step emits OTel spans. See exactly which prompt, which tool, which token caused the latency.

5. **Policy and identity**
   Mesh-level mTLS, policy-driven egress, audit logs. Your security team's existing controls just work.



Kubernetes native runtime

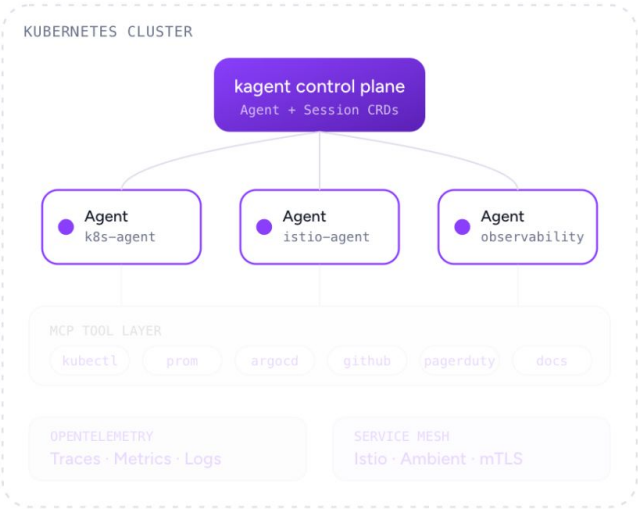
1. **Write an Agent CRD**
   Point at any LLM, attach the tools the agent needs (via MCP), and pick the cluster it should run in.

2. **kubectl apply -f agent.yaml**
   kagent provisions the runtime, wires up RBAC, and registers the agent with your service mesh.

3. **Tools and other agents**
   MCP servers expose your APIs as tools. A2A lets agents delegate to each other across the cluster.

4. **Traces, metrics, logs**
   Every step emits OTel spans. See exactly which prompt, which tool, which token caused the latency.

5. **Policy and identity**
   Mesh-level mTLS, policy-driven egress, audit logs. Your security team's existing controls just work.



Kubernetes native runtime

1 **Write an Agent CRD**

Point at any LLM, attach the tools the agent needs (via MCP), and pick the cluster it should run in.

2 **kubectl apply -f agent.yaml**

kagent provisions the runtime, wires up RBAC, and registers the agent with your service mesh.

3 **Tools and other agents**

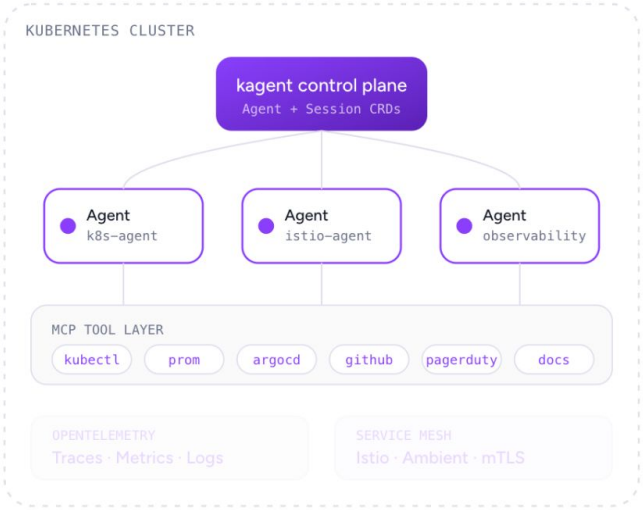
MCP servers expose your APIs as tools. A2A lets agents delegate to each other across the cluster.

4 **Traces, metrics, logs**

Every step emits OTel spans. See exactly which prompt, which tool, which token caused the latency.

5 **Policy and identity**

Mesh-level mTLS, policy-driven egress, audit logs. Your security team's existing controls just work.



Kubernetes native runtime

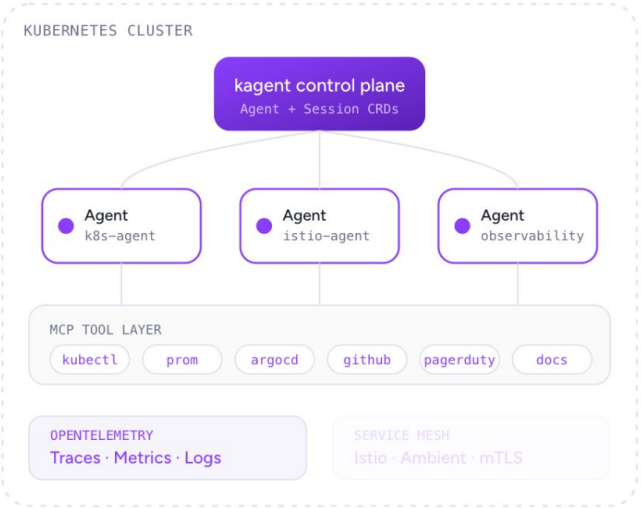
1. **Write an Agent CRD**
   Point at any LLM, attach the tools the agent needs (via MCP), and pick the cluster it should run in.

2. **kubectl apply -f agent.yaml**
   kagent provisions the runtime, wires up RBAC, and registers the agent with your service mesh.

3. **Tools and other agents**
   MCP servers expose your APIs as tools. A2A lets agents delegate to each other across the cluster.

4. **Traces, metrics, logs**
   Every step emits OTel spans. See exactly which prompt, which tool, which token caused the latency.

5. **Policy and identity**
   Mesh-level mTLS, policy-driven egress, audit logs. Your security team's existing controls just work.



Kubernetes native runtime

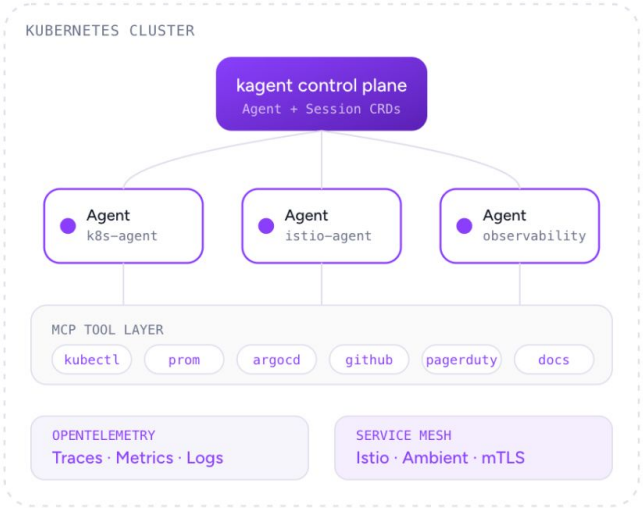
1. **Write an Agent CRD**
   Point at any LLM, attach the tools the agent needs (via MCP), and pick the cluster it should run in.

2. **kubectl apply -f agent.yaml**
   kagent provisions the runtime, wires up RBAC, and registers the agent with your service mesh.

3. **Tools and other agents**
   MCP servers expose your APIs as tools. A2A lets agents delegate to each other across the cluster.

4. **Traces, metrics, logs**
   Every step emits OTel spans. See exactly which prompt, which tool, which token caused the latency.

5. **Policy and identity**
   Mesh-level mTLS, policy-driven egress, audit logs. Your security team's existing controls just work.



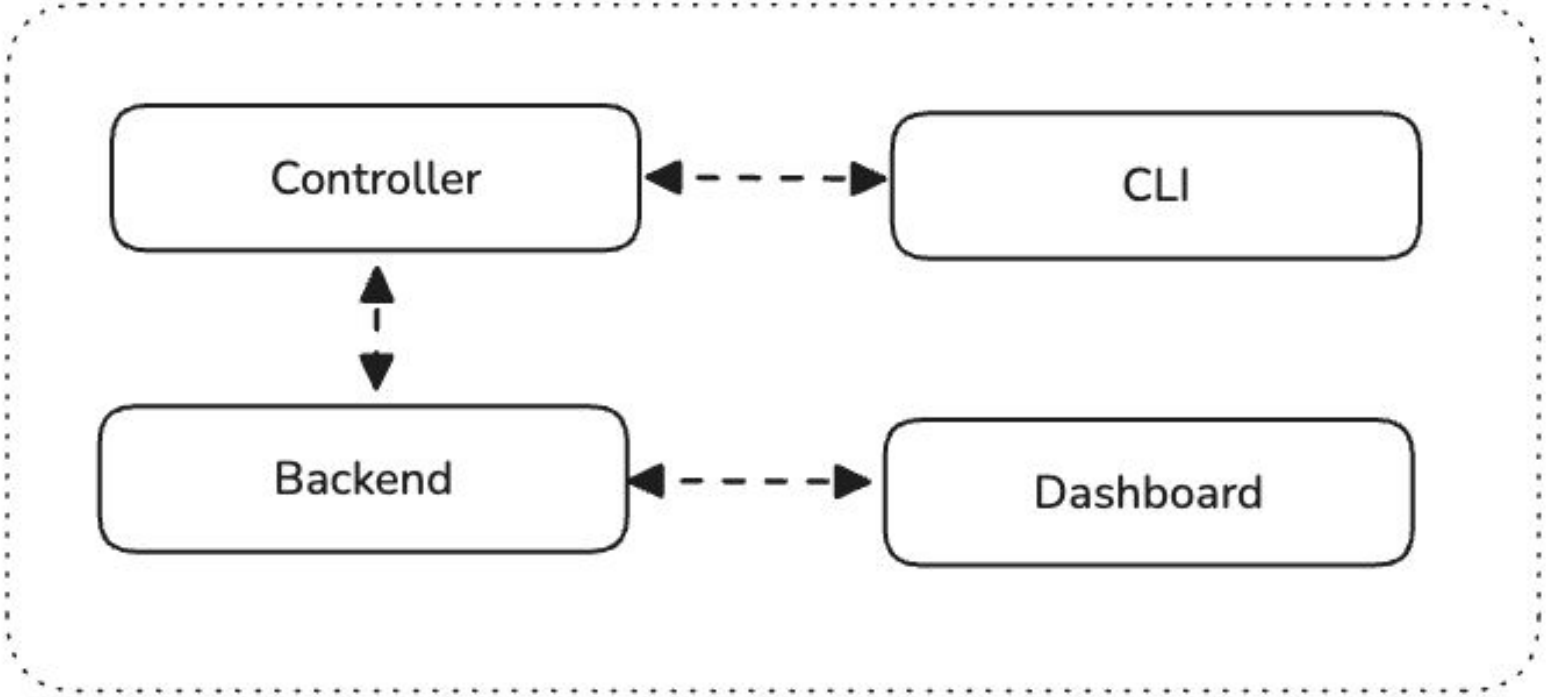
Everything as CRDs

For GitOps and AgentOps!

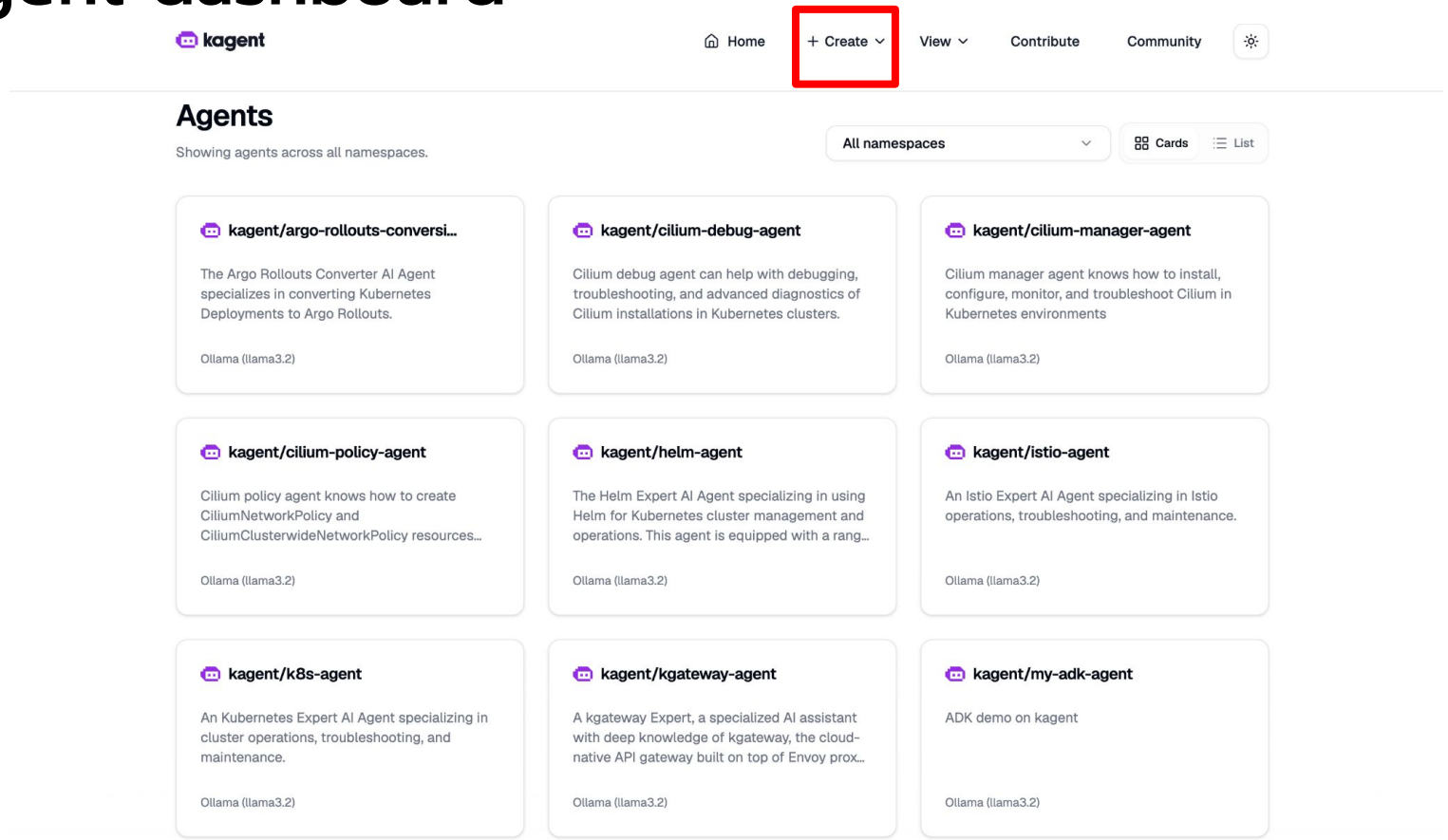
- **Agent CRD:** Core identity, logic, and operational parameters.
- **ModelConfig CRD:** LLM configuration, versioning, and connection details.
- **MCPServer CRD:** Declaration of authorized remote tool servers.
- **SandboxAgent CRD:** Isolated execution environments for untrusted code.

```yaml
# Sample Agent Resource
apiVersion: kagent.io/v1
kind: Agent
metadata:
name: sre-bot-cluster-ops
spec:
modelConfigRef: gpt4-prod
mcpServers:
- github-executor
- k8s-api-server
sandboxPolicy: strict
```

kagent



kagent dashboard



Demo

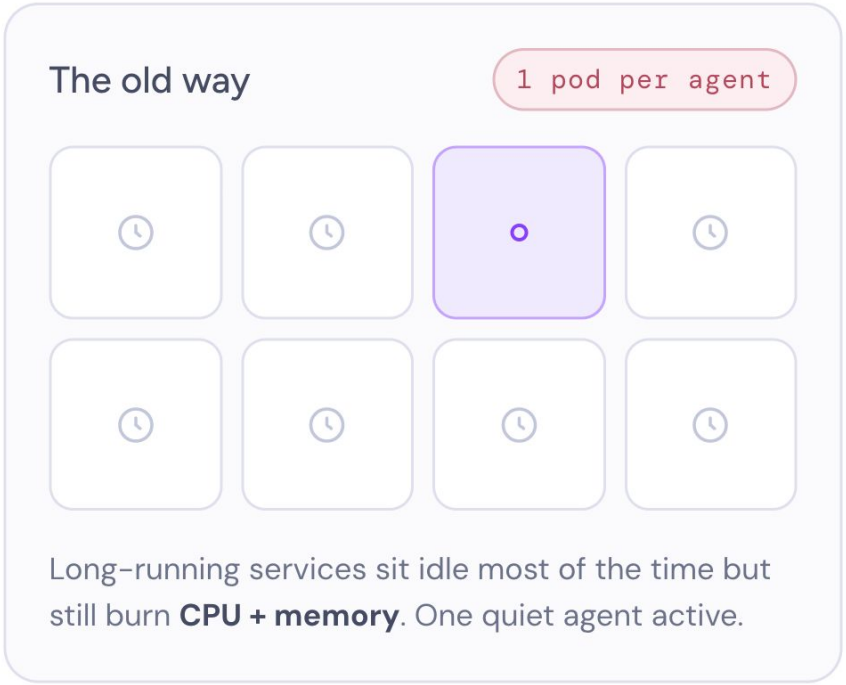
Agent Substrate

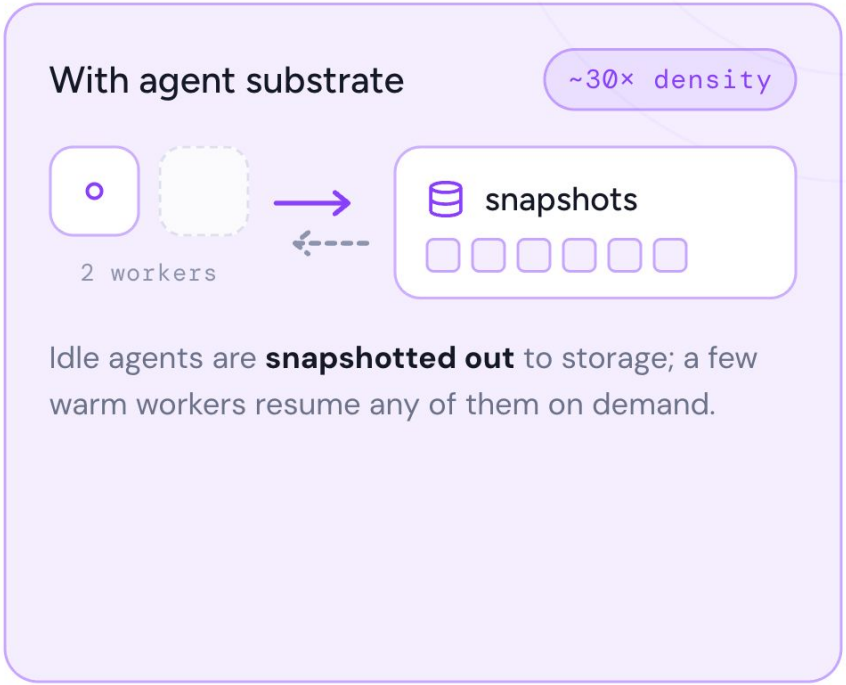
Agent Substrate: Problem statement

> “Agents are long-lived, bursty, and idle most of the time. We need lighter-weight isolation primitives: Firecracker microVMs, gVisor, Kata Containers and real lifecycle support: suspend, snapshot, resume, scale-to-zero with state preserved. “

https://www.solo.io/blog/agent-substrate-powers-kubernetes-agents-with-kagent

Agent Substrate





https://www.solo.io/blog/agent-substrate-powers-kubernetes-agents-with-kagent

Agent Substrate

workerpool.yaml

```yaml
apiVersion: ate.dev/v1alpha1
kind: WorkerPool
metadata:
  name: default
spec:
  replicas: 10
  ateomImage: >-
    ghcr.io/agent-substrate/ateom-gvisor:vX
status:
  replicas: 10   # actual count
```

autoscales with demand



https://www.solo.io/blog/agent-substrate-powers-kubernetes-agents-with-kagent

The Future of kagent: Agent Substrate

kagent → Agent Substrate

3 Insights

1. Agents dangerous → need sandbox



2. Need own filesystem



disk snapshot

3. Don't need 24/7 compute



idle clock + shared workers

Embrace Agent Substrate



Fast to resume • Cheap when idle • Sandboxed by default

250 agents on 8 pods

Sub-100ms resume

Biggest change for agents on Kubernetes

The Future of kagent: Agent Substrate

kagent → Agent Substrate

3 Insights

1. Agents dangerous → need sandbox



locked gVisor/microVM

2. Need own filesystem



disk snapshot

3. Don't need 24/7 compute



idle clock + shared workers

Embrace Agent Substrate



Fast to resume • Cheap when idle • Sandboxed by default

250 agents on 8 pods

Sub-100ms resume

Biggest change for agents on Kubernetes

3 Insights

1. Agents dangerous → need sandbox

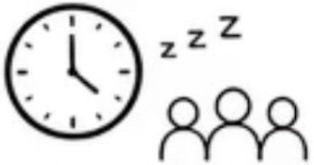


2. Need own filesystem



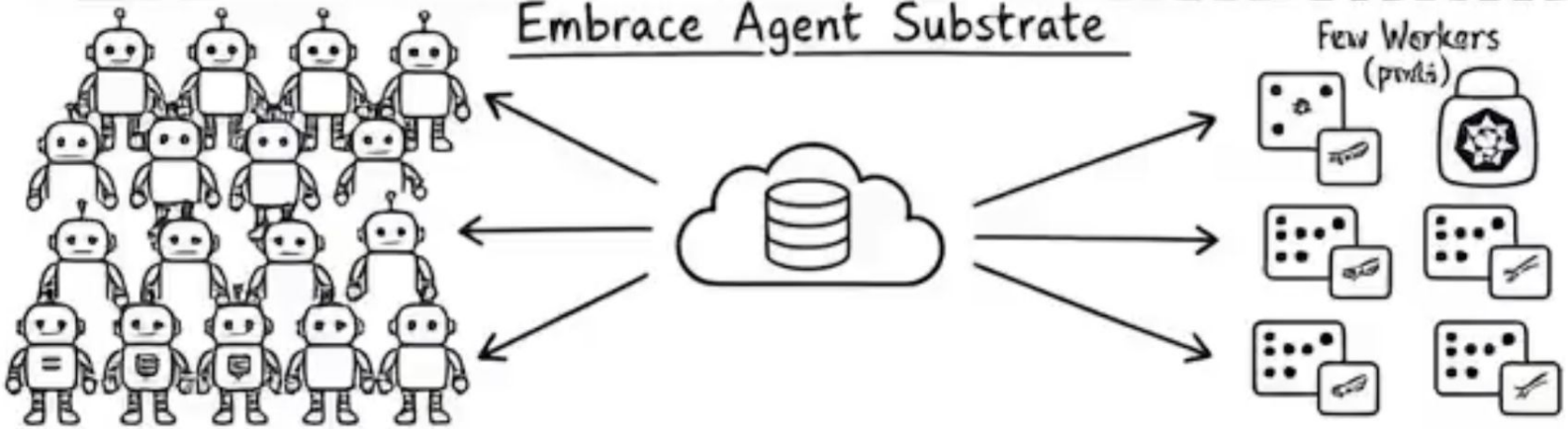
disk snapshot

3. Don't need 24/7 compute



idle clock + shared workers

Embrace Agent Substrate



Fast to resume • Cheap when idle • Sandboxed by default

Metrics: 250 agents on 8 pods

Biggest change for agents on Kubernetes

Summary

• Built on standards

An open stack, not another silo.

kagent doesn't reinvent the wheel. It composes the protocols that the agent ecosystem is converging on — so you can swap models, frameworks, and tools without rewriting your agents.



Model Context Protocol

Connect any tool — REST, gRPC, databases, internal APIs — through a single declarative spec.



Agent-to-Agent

Compose multi-agent systems with first-class delegation, hand-off, and shared memory.



OpenTelemetry

Every prompt, tool call, and token emits OTel traces — the same pipeline as the rest of your stack.



Kubernetes-native

Agents, sessions, and tools as CRDs. GitOps, RBAC, and admission controllers come for free.

Thank you



Fon - Kamolphan Liwprasert
Data Team Lead @ AIMET.tech